Chapter 20 — Pythonic Idioms, Gotchas and Interview Q&A

Pythonic code uses the language as its designers intended: readable, short without being clever, leaning on built-in protocols (the special methods Python's syntax calls — iteration, context managers, unpacking) instead of re-implementing them. Interviewers notice at once — `for i in range(len(xs))` or `if x == None` marks someone who learned Python from another language. The flip side is a handful of **gotchas**: behaviours that surprise even experienced developers (mutable default arguments, late-binding closures, shallow copies).

This chapter pulls the book together for review: idioms, then gotchas (each with demo and fix), then about 45 classic interview questions with short answers pointing to the chapter that teaches each topic. "Bank Qn" points to the same question in the Chapter 22 question bank, which adds many more. Scripts are in `code/`, run with Python 3.14 (all 3.12-compatible); outputs are real.

What you'll learn

#	Topic	Script
20.1	Pythonic idioms: EAFP, unpacking, `enumerate`/`zip`, comprehensions, truthiness, f-strings, context managers and `with open`, `pathlib`, and smaller habits	`01_idioms.py`
20.2	13 gotchas, each demonstrated and fixed	`02_gotchas.py`
20.3	Interview Q&A: language basics, data structures, functions, OOP, internals and memory, concurrency, typing/tooling	`03_interview_qa.py`

> The Zen of Python (`import this`, PEP 20; PEPs — *Python Enhancement Proposals* — are the numbered documents defining the language and its conventions) sums up the taste: *explicit is better than implicit, simple is better than complex, readability counts, errors should never pass silently, there should be one obvious way to do it.* PEP 8 is the style guide (4-space indents, `snake_case` functions, `CapWords` classes, lines ≲ 79–99 chars); `ruff` (linter) and `black` (formatter) enforce it automatically.

20.1 Pythonic idioms

What it is

An **idiom** is the community's conventional way to express a common task. Python idioms usually replace manual bookkeeping (indexes, flags, cleanup calls) with a language feature.

Why it matters

Fewer lines → fewer bugs (no off-by-one on an index you didn't need); others read it faster; and many idioms are faster because the work runs inside C built-ins (`sum`, `join`, comprehensions).

The idioms at a glance

Instead of…	Write…	Section
`if key in d: v = d[key] else: v = default`	`v = d.get(key, default)` or `try/except KeyError`	EAFP
`tmp = a; a = b; b = tmp`	`a, b = b, a`	Unpacking
`for i in range(len(xs)): x = xs[i]`	`for i, x in enumerate(xs)`	enumerate
`for i in range(len(a)): a[i], b[i]`	`for x, y in zip(a, b)`	zip
`out = []` + loop + `append`	`[f(x) for x in xs if cond(x)]`	Comprehensions
`if len(xs) == 0` / `if flag == True`	`if not xs` / `if flag`	Truthiness
`"Hi " + name + ", you are " + str(age)`	`f"Hi {name}, you are {age}"`	f-strings
`f = open(p); ...; f.close()`	`with open(p) as f:`	Context managers
`os.path.join(d, "x.txt")`	`Path(d) / "x.txt"`	pathlib
`s = ""` + `s += piece` in a loop	`"".join(pieces)`	Strings
`x == None`	`x is None`	Identity
`found = False` flag + loop	`for ... else:` or `any(...)`	Loops

EAFP vs LBYL

LBYL ("look before you leap") checks first: `if "port" in config: ...`. **EAFP** ("easier to ask forgiveness than permission") tries and handles failure: `try: port = int(config["port"]) except (KeyError, ValueError): port = 80`. Python favours EAFP because:

1. It avoids **race conditions** — between "check the file exists" and "open it", another process can delete it; opening inside `try` has no gap.
2. One `try` covers several failure modes (missing key *and* bad number).
3. The happy path (normal, no-error flow) stays unindented.

Use LBYL when failure is common and cheap to check (exceptions would be slow and noisy) or the check has no side effects to race with. `contextlib.suppress(FileNotFoundError)` is a tidy EAFP shortcut for "ignore this error". Never a bare `except:` (gotcha 11).

Unpacking

Any iterable unpacks into names; `*name` collects "the rest" into a list; `**` unpacks mappings.

```
a, b = b, a                                # swap, no temp variable
first, *middle, last = [10, 20, 30, 40, 50]
(x, y), z = (1, 2), 3                      # nested
merged = {**point, "z": 5}                 # dict unpacking (or point | {"z": 5}, 3.9+)
both = [*range(3), *"ab"]
_, ext = "report.final.pdf".rsplit(".", 1)  # _ = "don't care"
```

```
swap: 2 1 | star: 10 [20, 30, 40] 50 | nested: 1 2 3
dict merge: {'x': 3, 'y': 4, 'z': 5} | point | {'z': 5} -> {'x': 3, 'y': 4, 'z': 5} | list: [0, 1, 2, 'a', 'b']
rsplit + _: pdf
```

Also in `for name, score in pairs:` and calls (`f(*args, **kwargs)`, Chapter 7).

`enumerate` and `zip`

`enumerate(xs, start=1)` yields `(index, item)`; `zip(a, b)` yields same-position tuples and **stops at the shortest** input — `strict=True` (3.10+) raises `ValueError` on unequal lengths, a cheap guard against silent data loss. `zip(*rows)` "unzips" (transposes).

```python
for rank, name in enumerate(names, start=1):
    print(f"  {rank}. {name}")
dict(zip(names, scores))
list(zip(names, [1, 2], strict=True))   # ValueError
nums, letters = zip(*[(1, "a"), (2, "b")])
```

```
  1. ann
  2. bob
  3. cy
zip: [('ann', 91), ('bob', 78), ('cy', 85)] | dict(zip): {'ann': 91, 'bob': 78, 'cy': 85}
zip(strict=True): zip() argument 2 is shorter than argument 1
unzip: (1, 2) ('a', 'b')
```

Comprehensions and generator expressions

A **comprehension** builds a list/dict/set in one expression: `[expr for x in it if cond]`, `{k: v for ...}`, `{x for ...}`. A **generator expression** `(expr for x in it)` is lazy — pass it straight to `sum`, `any`, `max`, `"".join` to avoid building a list.

```python
[n * n for n in range(6)]                          # [0, 1, 4, 9, 16, 25]
[n for n in range(10) if n % 2 == 0]               # filter at the end
["even" if n % 2 == 0 else "odd" for n in range(4)] # if-else goes BEFORE for
{w: len(w) for w in ["hi", "hello"]}
sum(n * n for n in range(1_000_000))               # no million-item list
[x for row in [[1, 2], [3]] for x in row]          # loops read left-to-right
```

One `for` and one condition is ideal, two `for`s the maximum; past a line or two, write a loop. Don't use a comprehension for side effects (`[print(x) for x in xs]` builds a useless list).

Truthiness

Falsy: `None`, `False`, numeric zeros (`0`, `0.0`, `0j`), empty containers (`""`, `[]`, `()`, `{}`, `set()`, `range(0)`); everything else is truthy unless its class defines `__bool__`/`__len__` otherwise. So `if items:`, `if not name:`. Trap: `if count:` is also false for `0` — when `0`/`""` is valid and you mean "was it provided?", write `if count is not None:`. `and`/`or` return one of their **operands**, not a bool: `name or "anonymous"` defaults any falsy name.

f-strings

f-strings (3.6+) evaluate `{}` expressions and apply the **format mini-language** (width, alignment, number style) after a colon:

```python
print(f"{name:<8}|{price:>12,.2f}|{ratio:.1%}|{42:05d}|{255:#x}|{255:b}")
print(f"{price=}")          # 3.8+: prints the expression and its value
```

```
Widget  |    1,234.57|45.7%|00042|0xff|11111111
price=1234.5678
```

`<`/`>`/`^` align, the number is the width, `,` thousands separators, `.2f` decimals, `%` ×100, `x`/`b` hex/binary; `!r` uses `repr()`. For logging pass arguments lazily (`log.info("user %s", uid)`) so formatting happens only if emitted. Full detail: Chapter 4.

Context managers and `with open`

A **context manager**'s `__enter__` runs at the start of a `with` block and `__exit__` at the end — **even on an exception** (Chapter 12) — so cleanup of files, locks, transactions, temp dirs can't be forgotten.

```python
with open(path, "w", encoding="utf-8") as f:   # closed even on error
    f.write("line one\nline two\n")
with open(path, encoding="utf-8") as f:
    for line_no, line in enumerate(f, 1):      # stream: one line in memory at a time
        print(line_no, line.rstrip())

@contextmanager
def timer(label):
    start = time.perf_counter()
    try:
        yield                                   # the with-block body runs here
    finally:
        print(f"{label}: done")
```

Always pass `encoding="utf-8"` for text: the default comes from the platform's locale (e.g. often cp1252 on Windows) — "works on my machine" bugs. One `with` can hold several managers: `with open(a) as src, open(b, "w") as dst:`.

`pathlib`

`pathlib.Path` (Chapter 9): paths as objects, `/` joins, methods replace scattered `os.path` functions.

```python
base = Path(tmp)
notes = base / "notes" / "day1.txt"
notes.parent.mkdir(parents=True, exist_ok=True)
(base / "b.csv").write_text("a,b\n1,2\n", encoding="utf-8")
notes.name, notes.stem, notes.suffix     # 'day1.txt', 'day1', '.txt'
[p.name for p in base.glob("*.csv")]     # ['b.csv']
[p.relative_to(base).as_posix() for p in base.rglob("*.txt")]
```

```
name/stem/suffix: day1.txt day1 .txt
glob *.csv: ['b.csv']
rglob *.txt: ['notes/day1.txt']
exists: True | read_text: 'line one'
```

Smaller habits (all in `01_idioms.py`)

- **Chained comparisons:** `0 < x < 10`.
- **Conditional expression:** `"big" if x > 3 else "small"`.
- **`any` / `all` with generators** instead of flag loops; `max(xs, key=len, default=None)` for empty-safe maxima.
- **`for … else`**: `else` runs only if the loop didn't `break` — the "search failed" branch.
- **Walrus `:=`** (3.8+) assigns inside an expression: `if (m := len(s)) > 3:` — handy in `while` loops reading chunks.
- **`defaultdict` / `Counter`** instead of "if key not in d: d[key] = …".
- **`"sep".join(strings)`** to build strings.
- **`dataclass`** instead of hand-written `__init__` / `__repr__` / `__eq__` (Chapter 10).
- **`if __name__ == "__main__":`** so importing doesn't run script code (Chapter 8).

Common mistakes

- Comprehensions nested three deep — readable beats short.
- `except Exception: pass` — EAFP means *handling* a specific failure, not hiding all failures.
- Truthiness tests where `0` / `""` are valid.

In the interview

30-second answer: "Pythonic code uses the language's protocols: iterate directly with `enumerate` / `zip`, build collections with comprehensions, use truthiness, `with` for any resource, f-strings for formatting, `pathlib` for paths, and EAFP — try the operation and catch the specific exception, which also avoids check-then-use races."

- *EAFP or LBYL before opening a file?* → EAFP: `try: open(...) except FileNotFoundError:` — the file could vanish between check and open.
- *List comprehension or generator expression?* → Generator when consumed once (e.g. `sum`), especially for large data; list when you index, reuse or `len()` it.
- *`zip([1, 2, 3], "ab")`?* → `[(1, 'a'), (2, 'b')]` — stops at the shortest.

20.2 Top gotchas — demo and fix

Each block is printed by `02_gotchas.py`.

1. Mutable default arguments

```
def add_item_bad(item, bucket=[]):
    bucket.append(item)
    return bucket
```

```
bad : ['a', 'b'] ['a', 'b'] | defaults: (['a', 'b'],)
good: ['a'] ['b']
```

Why: defaults are evaluated **once**, when `def` runs, and stored on the function (`__defaults__`); every call omitting `bucket` shares one list. **Fix:** default to `None`, create inside: `if bucket is None: bucket = []` (same for `{}`, `set()`, any mutable). Linters flag it (ruff `B006`). Bank Q71, Q236.

2. Late-binding closures

```
fns = [lambda: i for i in range(3)]
[f() for f in fns]                          # [2, 2, 2]
fns = [lambda i=i: i for i in range(3)]     # [0, 1, 2]
```

Why: a closure references the *variable* `i`, looked up when **called** — after the loop, when it's 2. **Fix:** bind the current value with a default (`i=i`), `functools.partial(f, i)`, or a factory `def make(i): return lambda: i`. Bank Q81, Q237.

3. `is` vs `==`

```
a == b: True | a is b: False
x is None: True (the right way to test for None)
```

Why: `==` calls `__eq__` and compares **values**; `is` compares **identity** (`id(a) == id(b)`). **Fix:** `==` for values; `is` only for singletons — `None`, rarely `True`/`False`, and your own sentinels (unique "nothing was passed" markers, e.g. `_MISSING = object()`).

4. Integer caching

```
256 is 256: True | 257 is 257: False | 257 == 257: True
```

Why: CPython pre-creates −5…256 and reuses them; bigger runtime ints are separate objects. (Equal literals in one compiled block may be shared, so `a = 257; b = 257; a is b` can print `True` — an implementation detail either way.) Some strings are **interned** too (§17.3). **Fix:** never `is` for numbers or strings. Bank Q44, Q188.

5. Modifying a list while iterating

```
bad : [1, 2, 3]
good: [1, 3] | or iterate a copy: for n in nums[:]
dict: dictionary changed size during iteration
dict good: {'b': 2}
```

Why: the list iterator walks by index; removing shifts the rest left, so the next item is skipped (the second `2` survived). Dicts and sets detect size changes and raise `RuntimeError`. **Fix:** build a new collection, iterate a copy (`nums[:]`, `list(d)`), or collect keys and delete afterwards. Bank Q245.

6. Shallow copy

```
original after shallow edit: {'name': 'svc', 'ports': [80, 443]} | deep copy: {'name': 'svc', 'ports': [80]}
alias is config: True
```

Why: assignment copies nothing; a **shallow copy** (`d.copy()`, `list(x)`, `x[:]`, `copy.copy`) is a new outer container sharing inner objects; a **deep copy** (`copy.deepcopy`) copies recursively (memo dict handles cycles). **Fix:** `deepcopy` for nested data you'll change independently — or design with immutables (tuples, frozen dataclasses). For a 2-D list, `[row[:] for row in grid]`.

7. Shared mutable values: `dict.fromkeys` and `defaultdict` reads

```
fromkeys bad : {'a': [1], 'b': [1]}
fromkeys good: {'a': [1], 'b': []}
defaultdict after a read: {'missing': []} | use `'missing' in dd` or dd.get()
```

Why: `dict.fromkeys(keys, [])` stores the **same** list under every key (like `[[]] * n`); a `defaultdict` inserts a default on *any* `d[key]`, even a read, so `if dd["x"]:` grows the dict. **Fix:** `{k: [] for k in keys}`; test with `in` or `dd.get(key)` (doesn't insert). Bank Q55.

8. Class attributes vs instance attributes

```
bad : sam.tricks = ['roll']
good: sam.tricks = []
assignment shadows: wolf | canis familiaris | canis familiaris
```

Why: class-body attributes are shared by all instances; mutating one through an instance (`r.tricks.append`) changes it for everyone, while *assigning* (`r.species = "wolf"`) creates an instance attribute shadowing the class one for `r` only. **Fix:** per-instance mutable state in `__init__` (`self.tricks = []`); class attributes for constants and genuinely shared data. `dataclass` rejects `tricks: list = []` and requires `field(default_factory=list)`. Bank Q91, Q246.

9. Circular imports

```
bad : ImportError: cannot import name 'helper_a' from 'mod_a'
good: ba
```

Why: importing runs a module top to bottom. `mod_a` starts, hits `from mod_b import helper_b`, so `mod_b` runs — and its `from mod_a import helper_a` finds `mod_a` only *partially initialised*, without that name yet. (Message wording varies by version.) **Fixes, best first:**

1. Move the shared code into a third module both import (usually the real fix — a cycle signals muddled responsibilities).
2. `import mod_a` and use `mod_a.helper_a` at call time (the "good" version).
3. Import inside the function that needs it.
4. Type-hint-only imports: `from typing import TYPE_CHECKING` + `if TYPE_CHECKING: import ...`.

Bank Q169.

10. Floating point

```
0.1 + 0.2 = 0.30000000000000004 | == 0.3: False
math.isclose: True
Decimal('0.1') + Decimal('0.2') = 0.3 | Decimal(0.1) = 0.1000000000000000555...
Fraction(1, 10) + Fraction(2, 10) = 3/10
round(2.675, 2) = 2.67 | round(0.5), round(1.5) = 0 2
float('nan') == float('nan'): False
```

Why: `float` is IEEE-754 binary64 (the standard 64-bit binary format every modern CPU uses); most decimal fractions (0.1, 2.675) aren't exactly representable, so the nearest value is stored. `round` is **round-half-to-even** ("banker's rounding"), and `2.675` is really `2.67499999…`. `NaN` is unequal to everything, itself included. **Fix:** `math.isclose` (or an explicit tolerance); `decimal.Decimal` **from strings** for money; `fractions.Fraction` for exact rationals (one integer over another, like 1/3); integer cents for currency; `math.isnan` for NaN. Bank Q27, Q239.

11. Bare `except:`

```
bad : bare except swallowed KeyboardInterrupt
good: `except Exception` let KeyboardInterrupt through
```

Why: bare `except:` catches `BaseException`, including `KeyboardInterrupt` (Ctrl+C) and `SystemExit` (`sys.exit()`) — the program can't be stopped and real bugs vanish. **Fix:** catch the narrowest exception you can handle (`except KeyError:`); at the top level `except Exception:` and **log** (`logger.exception(...)`); re-raise with `raise` if you can't handle it. Bank Q152.

12. Shadowing built-ins

```
bad : 'list' object is not callable
good: ['a', 'b', 'c']
```

Why: lookup is Local → Enclosing → Global → **Built-in** (LEGB, Chapter 7), so a variable named `list`, `dict`, `str`, `id`, `input`, `sum`, `max`, `type`, `file` or `len` hides the built-in for that scope. **Fix:** descriptive names (`items`, `user_id`) or a trailing underscore (`type_`); ruff `A001` catches it. Don't name *files* after stdlib modules (`random.py`, `json.py`) either — they shadow the real module.

13. Bonus: return values that surprise

- `list.sort()`, `list.append()`, `random.shuffle()`, `dict.update()` modify in place and return `None` — `xs = xs.sort()` loses your list.
- `and` / `or` return an operand: `"" or "default"` → `'default'`, `0 and 1` → `0`.

Bank Q253.

Gotcha summary

Gotcha	Rule to remember
Mutable default	Defaults are evaluated once → use `None`
Late binding	Closures see variables, not values → `i=i`
`is` vs `==`	`is` only for `None`/sentinels
Int caching / interning	Implementation detail — never rely on it
Mutating while iterating	Build a new collection or iterate a copy
Shallow copy	Copies the container, shares the contents
`fromkeys` / `defaultdict`	Shared default object; reads insert
Class vs instance attrs	Mutable state goes in `__init__`
Circular imports	Restructure, or import the module not the name
Floats	`isclose`, `Decimal("…")`, `Fraction`, ints of cents
Bare `except`	Catch specific exceptions; log the rest
Shadowing built-ins	Don't name things `list`, `id`, `sum` …

In the interview

> **30-second answer:** "The classic ones: mutable default arguments are evaluated once, so use `None`; closures bind late, so capture loop variables with a default; `is` is identity, not equality; copying is shallow unless you `deepcopy`; don't mutate a collection while iterating it; floats are binary, so use `isclose` or `Decimal`; and never a bare `except`."

- *Why evaluate defaults once?* → They're part of the function object, created when `def` runs; re-evaluating per call would be slower and change the meaning of defaults referring to outer names.
- `a = [[0] * 2] * 2; a[0][0] = 1` → `a`*?* → `[[1, 0], [1, 0]]`.
- *What does bare* `except:` *catch that* `except Exception:` *doesn't?* → `KeyboardInterrupt`, `SystemExit`, `GeneratorExit` — `BaseException` subclasses outside `Exception`.

20.3 Python interview Q&A

Short answers to say in 20–40 seconds. "→ Ch N" is where the topic is taught in full; "Bank Qn" is the same question in Chapter 22. Demos come from `03_interview_qa.py`.

A. Language basics

Q1. Is Python compiled or interpreted? Both. CPython **compiles** source to **bytecode** (instructions for Python's own virtual machine, cached as `.pyc` in `__pycache__`), then its VM **interprets** it. No ahead-of-time compilation to machine code by default (3.13+ ships an experimental JIT — just-in-time compiler — off by default). `dis` shows the bytecode:

```
== bytecode: CPython compiles to bytecode, then interprets it
167           RESUME                   0
              LOAD_FAST_BORROW         0 (x)
              LOAD_SMALL_INT           1
              BINARY_OP                0 (+)
              RETURN_VALUE
```

(Opcodes change between versions.) Other implementations: **PyPy** (tracing JIT, often much faster for pure-Python loops), MicroPython (microcontrollers), historically Jython/IronPython. → Ch 1, 17 · Bank Q2

Q2. Python is dynamically and strongly typed — what does that mean? *Dynamic*: types belong to objects, not variables; names can be rebound to any type, checked at runtime. *Strong*: no silent conversion between unrelated types — `"1" + 1` raises `TypeError` (JavaScript gives `"11"`). → Ch 2 · Bank Q6

Q3. Mutable vs immutable types? Immutable: `int`, `float`, `bool`, `str`, `tuple`, `frozenset`, `bytes`. Mutable: `list`, `dict`, `set`, `bytearray`, most user classes. "Changing" an immutable (`s += "x"`) creates a new object and rebinds. Only hashable values can be dict keys/set members — a tuple *containing* a list is immutable itself, but its list can change, so it isn't hashable:

```
== mutable vs immutable, list vs tuple
   tuple holding a list: (1, [2, 3, 4])
   hash(tuple with list): unhashable type: 'list'
   sizes: list 88 bytes | tuple 72 bytes
```

→ Ch 2, 5 · Bank Q21

Q4. How are arguments passed — by value or by reference? By object reference ("call by sharing"): the parameter is a new name bound to the *same object*. Mutating it is visible to the caller; rebinding the parameter is not.

```
== pass by object reference
   after mutate: ['orig', 'changed']
```

(`lst.append` was visible; `lst = ["rebound"]` was not.) → Ch 7 · Bank Q70

Q5. `is` vs `==`? `==` compares values via `__eq__`; `is` compares identity. `is` only for `None` and sentinels (gotchas 3–4). → Ch 2 · Bank Q23

Q6. What does `if __name__ == "__main__":` do? `__name__` is `"__main__"` when the file runs directly, the import name when imported. The guard makes a file both an importable module and a script — and stops multiprocessing children (which re-import the main module under `spawn`) re-running your program. → Ch 8 · Bank Q166

Q7. Module vs package; what is `__init__.py`? A **module** is one `.py` file; a **package** a directory of modules. A *regular* package has `__init__.py` (run on first import, often exposing the public API); without it you get a *namespace package* (can span several directories). Imports are cached in `sys.modules`, so a module body runs once per process. → Ch 8 · Bank Q165

Q8. What is a virtual environment and why use one? An isolated directory with its own interpreter link and `site-packages` (where third-party packages install), made with `python -m venv .venv`. Each project pins its own dependency versions without conflicts or touching the system Python. Tools: `pip` + `requirements.txt`, or `uv` / `poetry` with `pyproject.toml` and lock files (exact versions of every package, for reproducible installs). → Ch 8 · Bank Q171

Q9. What's new that interviewers like to hear about? `match` (structural pattern matching — branching on data's *shape*, 3.10), `X | Y` unions (3.10), exception groups and `except*` (3.11), "Faster CPython" speed-ups (3.11+), type-parameter syntax `def f[T](x: T)` (3.12), the optional free-threaded (no-GIL) build (experimental 3.13, officially supported 3.14), and template strings (t-strings, 3.14 — like f-strings but producing a template object code can inspect and process safely before rendering). → Ch 1, 3, 13, 16 · Bank Q17

B. Data structures

Q10. List vs tuple?

	list	tuple
Mutable	yes	no
Hashable (dict key)	no	yes, if its items are
Typical meaning	many items of one kind (a collection)	a fixed record of positions (`(lat, lon)`)
Memory / speed	over-allocates to allow growth	smaller and slightly faster to create
Tuples also signal "this won't change". → Ch 5 · Bank Q45		

Q11. How is a list implemented? A dynamic array of pointers. Index O(1); `append` amortised O(1) (over-allocates, occasionally grows and copies); insert/delete at the front O(n). → Ch 5, 19 · Bank Q46

Q12. How does a dict work internally? A **hash table**: `hash(key)` picks a slot; collisions are resolved by **open addressing** (probing other slots in a pseudo-random sequence). Since CPython 3.6 the layout is *compact* — a small index table into a dense entry array — saving memory and preserving **insertion order**, guaranteed since 3.7. It resizes at about two-thirds full, keeping O(1) average (O(n) worst with many collisions). Keys must be hashable; equal objects must have equal hashes. → Ch 6 · Bank Q51

Q13. What makes an object hashable? What if I define `__eq__`? A `__hash__` returning an int that never changes over its lifetime, consistent with `__eq__` (equal ⇒ same hash). Defining `__eq__` without `__hash__` sets `__hash__ = None` → unhashable. `@dataclass(frozen=True)` generates both. → Ch 6, 10 · Bank Q52, Q102

Q14. Set vs list for membership? Set: O(1) average `in`, no duplicates, unordered. List: O(n) scan, keeps order and duplicates. Convert to a set for repeated membership tests. → Ch 6, 19 · Bank Q58

Q15. Shallow vs deep copy? Shallow copies the container, sharing elements; deep copies recursively.

```
== copy vs deepcopy
   shallow: {'tags': ['a', 'b']} | deep: {'tags': ['a']}
```

Customise with `__copy__`/`__deepcopy__`. → Ch 5, gotcha 6 · Bank Q60

Q16. `array` vs `list` vs NumPy array? `list` holds pointers to arbitrary objects; `array.array` stores raw C numbers of one type compactly; a **NumPy** `ndarray` is a typed contiguous block *with* C-implemented vectorised operations — orders of magnitude faster for numeric work. → Ch 5, 18 · Bank Q66

Q17. What's in `collections`? `Counter` (counting), `defaultdict` (auto-defaults), `deque` (O(1) both ends), `namedtuple` (tuple with named fields), `OrderedDict` (order-aware equality, `move_to_end` — handy for LRU caches), `ChainMap` (layered lookups). → Ch 6 · Bank Q61

C. Functions

Q18. What are `*args` and `**kwargs`? In a definition `*args` collects extra positional arguments into a **tuple**, `**kwargs` extra keyword arguments into a **dict** (names are convention). At a call, `*`/`**` **unpack** an iterable/mapping. Signature order: positional-only (before `/`, can't be passed by name), normal, `*args`, keyword-only (must be `name=value`), `**kwargs`.

```
== *args / **kwargs
   show(1, 2, a=3) -> ((1, 2), {'a': 3}) | show(*[1, 2], **{'b': 4}) -> ((1, 2), {'b': 4})
```

Typical use: wrappers/decorators forwarding arguments unchanged. → Ch 7 · Bank Q69

Q19. What is a closure? A function that remembers variables from its enclosing scope after that function returned. They live in *cells* (`f.__closure__`); `nonlocal` rebinds them. Closures bind late (gotcha 2). → Ch 7 · Bank Q80

Q20. Explain LEGB and `global`/`nonlocal`. Lookup order: **L**ocal, **E**nclosing functions, **G**lobal (module), **B**uilt-ins. Assigning anywhere in a function makes the name local for the whole function (hence `UnboundLocalError` if read first). `global x` rebinds the module variable; `nonlocal x` the nearest enclosing function's. → Ch 7 · Bank Q77

Q21. What is a decorator? Write one with arguments. A callable taking a function and returning a replacement; `@d` above `def f` means `f = d(f)`. With arguments, one more layer: `@retry(3)` calls `retry(3)`, which returns the real decorator. Always `functools.wraps`, keeping `__name__`, `__doc__`, `__wrapped__`.

```python
def retry(times):
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(1, times + 1):
                try:
                    return func(*args, **kwargs)
                except ValueError:
                    print(f"   attempt {attempt} failed")
            raise RuntimeError("gave up")
        return wrapper
    return decorator
```

```
== decorators
   attempt 1 failed
   attempt 2 failed
    ok | name kept by wraps: flaky | Fails twice, then works.
```

Real uses: `@app.get("/")` (FastAPI/Flask routing), `@functools.cache`, `@pytest.fixture`, `@login_required` (Django). → Ch 12 · Bank Q135, Q138, Q274

Q22. Iterator vs iterable vs generator? An **iterable** has `__iter__` returning an iterator (lists, dicts, files). An **iterator** has `__next__` (next item or `StopIteration`) and `__iter__` returning itself; single-pass. A **generator** is an iterator from a `yield` function (or generator expression), its state saved between `next` calls. Every generator is an iterator, not vice versa.

```
== generators vs iterators
   iter(gen) is gen: True | values: [3, 2, 1] | second pass: []
   list 85176 bytes vs generator 208 bytes
```

The second pass is empty — iterators exhaust; the generator's size doesn't depend on how many items it yields. → Ch 11 · Bank Q121

Q23. `yield` vs `return`; what is `yield from`? `return` ends the function with one value; `yield` produces a value and **pauses** until the next `next()`. `yield from sub` delegates to a sub-iterator (re-yielding its values, passing `send()`/`throw()` through). `send(value)` pushes values in — the historical basis of coroutines (`async`). → Ch 11 · Bank Q123

Q24. What is a lambda and when shouldn't you use it? An anonymous single-expression function, `lambda x: x * 2` — good for short `key=` arguments. Don't assign one to a name (`def` gives a proper traceback name) or cram logic into it. → Ch 7 · Bank Q74

Q25. `map`/`filter` vs comprehensions? Both lazy in Python 3 (`map`/`filter` return iterators). Comprehensions usually read better, especially when a `lambda` would be needed; `map(str, xs)` with an existing function is fine. → Ch 11 · Bank Q76

D. Object-oriented programming

Q26. `__init__` vs `__new__`? `__new__(cls, …)` is a static method that **creates** and returns the instance; `__init__(self, …)` **initialises** it and returns `None`. Override `__new__` only for immutable subclasses (`int`, `str`, `tuple`), singletons/instance caching, or metaclass work (Q35). `__init__` still runs on every call:

```
== __new__ vs __init__
   __new__: creating the object
   __init__: setting value=1
   __init__: setting value=2
   same object: True | value: 2
```

→ Ch 10 · Bank Q90

Q27. `@staticmethod` vs `@classmethod` vs instance method?

Kind	First parameter	Can access	Typical use
Instance method	`self`	instance and class	normal behaviour
`@classmethod`	`cls`	the class (and subclasses correctly)	alternative constructors (`dict.fromkeys`, `datetime.fromtimestamp`, `from_fahrenheit`)
`@staticmethod`	none	nothing implicit	a helper that logically belongs to the class

```
== classmethod / staticmethod / property / __slots__
    Temperature(100.0) | is_freezing(-3): True
```

→ Ch 10 · Bank Q92

Q28. What does `@property` do? Makes a method a computed attribute with optional setter/deleter — start with a plain attribute, add validation later without changing callers (`t.celsius = -300` → `ValueError: below absolute zero`). No need for Java-style getters/setters. → Ch 10 · Bank Q93

Q29. What is the MRO and how does `super()` work? The **Method Resolution Order** — the order classes are searched for an attribute — comes from **C3 linearisation** (merging all parent orders consistently): children before parents, base order left-to-right preserved. `super()` means "the next class in the MRO of the *instance's* class", not "my parent", so cooperative multiple inheritance (each parent calls `super()`; "diamonds" share a grandparent) calls each class once:

```
== MRO / super()
   D.__mro__: ['D', 'B', 'C', 'A', 'object'] | D().hello(): D>B>C>A
```

`B`'s `super()` went to `C`, not `A`, because the instance is a `D`. → Ch 10 · Bank Q97

Q30. `__str__` vs `__repr__`? `__repr__` is for developers — unambiguous, ideally code that recreates the object (`Temperature(100.0)`); used by the REPL, debuggers and inside containers. `__str__` is for users

(`print`, `str()`, f-strings), falling back to `__repr__`. Define `__repr__` first. → Ch 10 · Bank Q99

Q31. What are dunder (magic) methods? `__x__` methods Python calls for syntax and built-ins — the **data model**: `len(x)` → `__len__`, `a + b` → `__add__`, `x[i]` → `__getitem__`, `for` → `__iter__`, `with` → `__enter__`/`__exit__`, `==` → `__eq__`, `bool(x)` → `__bool__`, calling → `__call__`. They make classes behave like built-ins. → Ch 10 · Bank Q100

Q32. What does `__slots__` do? Declares fixed attribute names stored in a compact array instead of a per-instance `__dict__` — less memory per object (millions of small objects) and slightly faster access, but no new attributes:

```
__slots__ blocks new attributes: 'Temperature' object has no attribute 'colour' and no __dict__ for setting new attributes
```

`@dataclass(slots=True)` (3.10+) generates it. → Ch 10, 17 · Bank Q111

Q33. Abstract base classes vs duck typing vs `Protocol`? Duck typing: if it has the method, it works — no inheritance. An **ABC** (`abc.ABC` + `@abstractmethod`) enforces at instantiation that subclasses implement methods (nominal: must inherit). A `typing.Protocol` describes required methods structurally, checked by static type checkers without inheritance — "duck typing mypy can verify" (`Square` satisfies `HasArea` without inheriting it). → Ch 10, 13 · Bank Q107, Q108

Q34. What is a dataclass? `@dataclass` generates `__init__`, `__repr__`, `__eq__` from annotated fields; options add ordering (`order=True`), immutability + hashing (`frozen=True`), `slots=True`, and `field(default_factory=list)` for mutable defaults. Plain data records; Pydantic when you need runtime validation/parsing. → Ch 10 · Bank Q109

Q35. What are descriptors and metaclasses (briefly)? A **descriptor** is a class attribute with `__get__`/`__set__`/`__delete__` that routes instance attribute access; `property`, methods (binding `self` via `__get__`), `classmethod`, `staticmethod` are descriptors. A **metaclass** is the class of a class (default `type`), controlling class *creation*; ORMs (Django models), `abc.ABCMeta` and `enum.EnumMeta`/`EnumType` use them. A class decorator or `__init_subclass__` is usually simpler. → Ch 12 · Bank Q118, Q119

E. Errors, resources, memory and internals

Q36. Explain `try / except / else / finally` and `raise … from`. `except` handles matching exceptions; `else` runs only if `try` raised nothing (put code that shouldn't be protected there); `finally` always runs (cleanup). `raise NewError(...) from err` chains, storing the cause in `__cause__` so the traceback shows both. Custom exceptions subclass `Exception`. → Ch 9 · Bank Q151

Q37. How does a context manager work? `with obj as x:` calls `obj.__enter__()` (return value bound to `x`), runs the block, then always `obj.__exit__(exc_type, exc, tb)`; a true return suppresses the exception. `contextlib.contextmanager` builds one from a generator: before `yield` is enter, after it (in `finally`) is exit. → Ch 12 · Bank Q143

Q38. How does Python manage memory?

1. **Reference counting**: freed **immediately** at count zero (deterministic cleanup).
2. A generational **cyclic garbage collector** (`gc`) periodically frees groups of objects that only reference each other.
3. Small objects come from **pymalloc**, CPython's pooling allocator, avoiding many small `malloc` (C's general allocator) calls.

```
== memory: reference counting + cycle collector
   refcount (includes getrefcount's own arg): 2
   freed immediately by refcounting: True
   cycle alive before gc.collect(): True
   cycle freed after gc.collect(): True
```

"Leaks" are usually lingering references: growing global caches/lists, unbounded `lru_cache`s, closures holding big objects. Diagnose with `tracemalloc`. → Ch 17 · Bank Q182, Q185

Q39. What is pickling? Is it safe? `pickle` **serialises** objects to bytes (`dumps`/`dump`) and back (`loads`/`load`); `multiprocessing` uses it between processes, and scikit-learn models are saved with it

(joblib builds on it). Functions and classes pickle *by reference* (module + name), so lambdas and local functions can't:

```
== pickling
   round trip: {'model': 'v1', 'weights': [0.1, 0.2]} | 61 bytes
   pickle a lambda: PicklingError
```

Never unpickle untrusted data — loading can execute arbitrary code. For exchange use JSON (or Parquet / `safetensors` — a safe model-weights format — for arrays). → Ch 9 · Bank Q162

Q40. How do you find and fix slow Python code? Measure first: `cProfile`/`pstats` (which functions), `timeit` (micro-benchmarks), `tracemalloc` (memory). Then: better algorithms/data structures (sets for membership, deque for queues), built-ins or NumPy vectorisation, caching (`functools.cache`), fewer attribute lookups in hot loops — and only then PyPy, Cython/Rust extensions, or processes. → Ch 17 · Bank Q192

F. Concurrency

Q41. What is the GIL? The **Global Interpreter Lock** is a mutex in CPython letting only one thread execute bytecode at a time, keeping reference counting and interpreter internals simple and fast single-threaded. Consequences:

- **CPU-bound** pure Python does **not** speed up with threads — use `multiprocessing`/`ProcessPoolExecutor` (one GIL each) or C extensions that release the GIL (NumPy, for many operations).
- **I/O-bound** code *does* benefit — the GIL is released while waiting.
- It doesn't make your code thread-safe: `counter += 1` is several bytecodes and can race — use `threading.Lock`.

```
== GIL: threads vs processes on CPU-bound work
   GIL enabled: True
   CPU-bound x4  serial 0.95s | 4 threads 1.02s | 4 processes 0.56s
```

(Machine-dependent; the pattern matters.) Processes helped, less than 4× due to start-up and data transfer. PEP 703's **free-threaded** build (`python3.13t`/`3.14t`) removes the GIL; it's optional, the default build keeps it. → Ch 16 · Bank Q197

Q42. Threads vs processes vs asyncio — which when?

Workload	Choose	Why
CPU-bound (math, parsing, image processing)	`multiprocessing` / `ProcessPoolExecutor`	true parallelism, one GIL per process
I/O-bound, moderate concurrency, blocking libraries	`threading` / `ThreadPoolExecutor`	GIL released during I/O; simple
I/O-bound, thousands of connections, async libraries	`asyncio`	one thread, very cheap tasks
Numeric arrays	NumPy/vectorisation	C loops, often release the GIL
→ Ch 16 · Bank Q201		

Q43. How does `asyncio` work? A single-threaded **event loop** runs **coroutines** (`async def`). At each `await` on something not ready, the coroutine yields to the loop, which runs another ready task — **cooperative multitasking** (tasks hand over control voluntarily). `asyncio.gather` runs many concurrently:

```
== asyncio: many waits overlap on one thread
   10 x 0.2s waits concurrently -> 0.20s, ['req0', 'req1', 'req2']...
```

Ten 0.2 s waits took 0.2 s total. Pitfall: a blocking call (`time.sleep`, `requests.get`, heavy CPU) in a coroutine freezes the loop — use async libraries (`httpx`, `aiohttp`) or `await asyncio.to_thread(func)`. → Ch 16 · Bank Q211

G. Typing and tooling

Q44. Are type hints enforced? No — stored in `__annotations__`, ignored at runtime:

```
== typing is not enforced at runtime; Protocol = structural typing
   double('ab') runs anyway: abab
   total_area: 13
```

Static checkers (**mypy**, **pyright**) and IDEs use them before running; **Pydantic** and **FastAPI** read them to validate at runtime. Vocabulary: `list[int]`, `dict[str, float]`, `X | None` (= `Optional[X]`), `Callable[[int], str]`, `TypeVar`/generics (`def first[T](xs: list[T]) -> T`, 3.12), `Protocol`, `TypedDict`, `Literal`, `Any`. → Ch 13 · Bank Q174

Q45. How do you test Python code? `pytest`, the de-facto standard: plain `assert`, fixtures for setup, `@pytest.mark.parametrize` for many cases; `unittest` is the stdlib xUnit style (class-based, like JUnit). `unittest.mock` (`patch`, `Mock`) replaces external dependencies, `coverage` measures what ran, `hypothesis` does property-based tests (state a rule; the library generates inputs to break it). → Ch 15 · Bank Q226

Q46. What is monkey patching? Replacing a module or class attribute at runtime (`module.func = fake`). Fine in tests — `unittest.mock.patch("pkg.mod.name")` does it safely and restores the original — but dangerous in production: global and invisible. Patch where the name is **looked up**, not where defined. → Ch 12, 15 · Bank Q147, Q229

Rapid-fire round

Question	One-line answer
`range` in Python 3?	A lazy, immutable sequence; O(1) `len` and `in` for ints.
Are strings mutable?	No; build with `join`, or `io.StringIO`.
What does `pass` do?	Nothing — a placeholder where a statement is required.
`append` vs `extend`?	`append(x)` adds one item; `extend(it)` adds each item of an iterable.
`sorted` vs `.sort()`?	New list vs in-place (returns `None`); both stable Timsort.
`del` vs `remove` vs `pop`?	By index/slice/name; by value (first match); by index returning the item.
What is `None`?	The single instance of `NoneType`, meaning "no value"; test with `is None`.
Why `self`?	Methods receive the instance explicitly; `obj.m()` is `Class.m(obj)`.
What is `__call__`?	Makes instances callable like functions.
What is a namedtuple?	A tuple subclass with named fields: lightweight immutable records.
`@functools.cache` vs `lru_cache`?	Unbounded vs size-limited memoisation.
`copy.copy` on a tuple of ints?	Returns the same tuple (immutable — no need to copy).
What is `__pycache__`?	Cached bytecode (`.pyc`) to skip recompiling unchanged modules.
What is `*` alone in a signature?	Everything after it is keyword-only.
What does `/` in a signature mean?	Everything before it is positional-only.
`match` statement?	Structural pattern matching (3.10): destructures sequences, mappings, classes.
What does `enumerate` return?	A lazy iterator of `(index, item)` tuples.

How to answer well

1. **Lead with a one-sentence definition**, then the mechanism, then a use case or trade-off.
2. **Show you know the edges**: "…except the free-threaded build", "…O(1) on average, O(n) worst case".
3. **Offer a tiny example** ("`@app.get` in FastAPI is a decorator").

4. **Say "I'd check the docs"** rather than invent a detail — accuracy beats bluffing.

Self-check

1. "The first item and the rest" in one line? → `first, *rest = items`.
2. Why pass `encoding="utf-8"` to `open`? When does a loop's `else` run? → The default text encoding is platform-dependent; when the loop ends without `break`.
3. `def f(x, seen=set())` — what's wrong? → `seen` is shared between calls.
4. Compare two floats for "equality"? → `math.isclose(a, b)` (with `abs_tol` near zero).
5. Why doesn't threading speed up a CPU-bound pure-Python loop in standard CPython? → The GIL lets one thread run bytecode at a time.
6. What does `super()` resolve to in a diamond? → The next class in the instance's MRO.
7. When is `__new__` worth overriding? → Immutable subclasses, singletons/caching, metaclass-level control.
8. The two parts of CPython's memory reclamation? → Reference counting + cyclic GC.
9. Is `pickle.loads` on a user upload safe? → No — it can run arbitrary code.

Chapter TL;DR

- **Idioms:** EAFP with specific exceptions; unpacking (`a, b = b, a`, `first, *rest`); `enumerate` / `zip(strict=True)`; comprehensions and generator expressions; truthiness (`if not xs`, but `is not None` when 0 is valid); f-strings with format specs and `{x=}`; `with` for every resource; `open(..., encoding="utf-8")`; `pathlib.Path`; `"".join`; `for … else`; `:=`.
- **Gotchas:** mutable defaults (use `None`), late-binding closures (`i=i`), `is` vs `==`, int caching/interning (don't rely), mutating while iterating (new list or copy), shallow vs deep copy, `dict.fromkeys` / `defaultdict` surprises, class vs instance attributes, circular imports (restructure / import the module), float precision (`isclose`, `Decimal("…")`), bare `except` (catch specific), shadowing built-ins.
- **Interview core:** bytecode + interpreter; dynamic + strong typing; call by object reference; list = dynamic array, dict = ordered hash table; hashable = immutable + `__hash__` / `__eq__`; decorators (`f = d(f)`), closures, generators; `__new__` creates, `__init__` initialises; static vs class methods; MRO/C3 and `super()`; dunder methods; `__slots__`; ABC vs Protocol; refcounting + cycle GC; pickle is unsafe for untrusted data; GIL → processes for CPU, threads/asyncio for I/O; type hints checked by tools, not at runtime.